```
      .attr("cy", function(d) {return scale(d.y);})
      .attr("r", 20)
      .style("fill", "pink");
}
```

The results in figure 10.1 show how the grid function has correctly appended x and y coordinates to draw the tweets as circles on a grid.

The benefit of building this as a layout is that if we add more data to it, it automatically adjusts and allows us to use transitions to animate that adjustment. To do this, we need more data. Listing 10.4 includes a few lines to create data that represents our new tweets. We also use the `.concat()` function of an array in native JavaScript that, when given the state shown in figure 10.1, should produce the results in figure 10.2.

Listing 10.4 Update the grid with more elements

```
var fakeTweets = [];
for (var x = 0;x < 12;x++) {
     var tweet = {id: x, content: "Fake Tweet #" + x};
     fakeTweets.push(tweet);
}

var doubledArray = data.tweets.concat(fakeTweets);
var newGriddedData = grid(doubledArray);

d3.select("svg").selectAll("circle")
     .data(newGriddedData)
     .enter()
     .append("circle")
     .attr("cx", 0)
     .attr("cy", 0)
     .attr("r", 20)
     .style("fill", "darkred");

d3.select("svg").selectAll("circle")
     .transition()
     .duration(1000)
     .attr("cx", function(d) {return scale(d.x)})
     .attr("cy", function(d) {return scale(d.y)})
```

Creates 12 new fake tweets

Combines the original dataset with our new dataset

Adds any new tweets at 0,0

Moves all tweets (old and new) to their newly computed positions

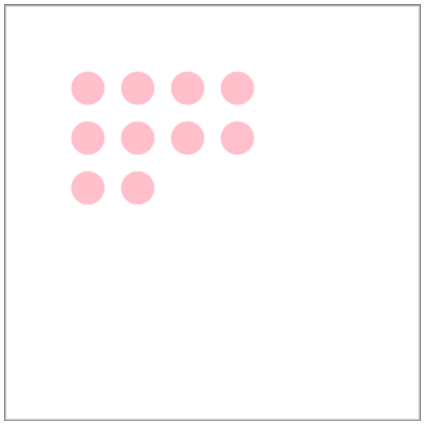


Figure 10.1 The results of our `makeAGrid` function that uses our new `d3.layout.grid` to arrange the data in a grid. In this case, our data consists of 10 tweets that are each represented as a pink circle laid out on a grid.